Final Report of Traineeship Program 2025
On
“Analysis of Chemical Components Project Proposal ”

MedTourEasy

28th December 2025

Submitted by
Priyanka verma
(Data Analytics Intern)

ACKNOWLEDGMENT

I would like to express my sincere gratitude to **MedTourEasy** for providing me with the opportunity to undertake this project and for their valuable support throughout its completion. I am grateful to Support team of MTE for their guidance, encouragement, and constructive feedback, which were essential to the successful execution of this project.

I also acknowledge the **Google Data Analytics Professional Certificate platform** for its structured learning resources and practical guidance, which contributed significantly to the direction and quality of this work.

I thank the team at MTE for their cooperation and support during the project period.

I also extend my thanks to Ira Skills and my faculty members for their academic guidance and continued encouragement with full support.

TABLE OF CONTENTS

ABSTRACT

The cosmetic industry produces a wide range of daily-use products that contain diverse chemical components, making product selection complex for consumers with specific skin requirements. This project focuses on the analytical evaluation of cosmetic products, with particular emphasis on moisturizers, to identify suitable products based on ingredient composition, skin type compatibility, and pricing attributes. Using a structured dataset comprising product names, brands, ingredient lists, labels, prices, and supported skin types, data analysis techniques were applied to extract meaningful insights.

The analysis was conducted using Python and key data science libraries, including Pandas and NumPy for data preprocessing and filtering, Matplotlib for exploratory data visualization, and scikit-learn for basic feature encoding and modeling. The dataset was first filtered to isolate moisturizer products, followed by further segmentation based on specific skin types such as dry, oily, or sensitive skin. Ingredient-level analysis was performed to identify products free from potentially harmful chemical components, thereby supporting safer product recommendations.

The results demonstrate how data-driven filtering and analytical techniques can assist in identifying optimal cosmetic products tailored to individual skin needs. This project highlights the practical application of data analytics in the cosmetic domain and illustrates how analytical methods can support informed decision-making for both consumers and businesses.

1. INTRODUCTION

1.1 About the company

MedTourEasy, a global healthcare company, provides you the informational resources needed to evaluate your global options. MedTourEasy provides analytical solutions to our partner healthcare providers globally.

1.2 About the project

- This project addresses the challenge consumers face when selecting cosmetic products, particularly those with **dry or sensitive skin**, due to the complexity of interpreting ingredient lists without a background in chemistry. Cosmetic labels contain extensive chemical component information, yet this data remains largely inaccessible to average consumers. Instead of relying on trial and error, this project leverages **data science and machine learning techniques** to support informed and safer cosmetic product selection.
- The core focus of the project is the **analysis of chemical components in cosmetic products** and the development of a **content-based recommendation system**, where the "content" refers specifically to ingredient composition. The dataset consists of ingredient lists for **1,472 cosmetic products sourced from Sephora**, along with associated product metadata. Ingredient text data is processed using **word embedding techniques** to capture semantic relationships between chemical components commonly used in dry-skin formulations.
- To explore similarity patterns among products, a machine learning–based dimensionality reduction technique, **t-distributed Stochastic Neighbour Embedding (t-SNE)**, is applied. The resulting ingredient

similarity space is visualized using **Bokeh**, enabling interactive exploration of product clusters based on shared chemical composition. Recommendations are generated by identifying products with ingredient profiles similar to those known to be suitable for dry skin.

- This project demonstrates how **chemical component analysis forms the foundation of cosmetic recommendations**, highlighting the practical application of natural language processing and machine learning in the cosmetics and personal care domain.

1.3 Objectives

The key objectives of this project are:

1. To analyse and preprocess cosmetic ingredient lists as textual chemical component data.
2. To identify **common and beneficial chemical components** used in dry-skin cosmetic products.
3. To build a **content-based recommendation system** using ingredient composition as the primary feature.
4. To apply **word embedding techniques** to represent chemical components in a meaningful numerical form.
5. To visualize cosmetic product similarity using **t-SNE** for dimensionality reduction.
6. To create an **interactive visualization** that enables intuitive exploration of ingredient-based similarity.
7. To support safer and more informed cosmetic product selection using data-driven insights.

1.4 Project Deliverables

The final deliverables of this project include:

1. Cleaned and pre-processed cosmetic dataset with structured ingredient data.
2. Feature representations of chemical components using word embeddings.

3. Ingredient similarity analysis and clustering results.
4. t-SNE visualizations of cosmetic product similarity.
5. Interactive visualization dashboard using Bokeh.
6. A content-based recommendation logic for dry-skin cosmetic products.
7. Final project report documenting methodology, results, and conclusions.
8. Source code repository (Python notebooks/scripts).

2. Methodology

2.1 (Flow of the Project)

Step 1: Data Collection

- Import cosmetic product dataset containing ingredient lists and product metadata.
- Focus on products relevant to dry skin.

Step 2: Data Cleaning & Preprocessing

- Handle missing values and inconsistent ingredient naming.
- Normalize and tokenize ingredient text.
- Remove irrelevant symbols and standardize chemical terms.

Step 3: Feature Engineering (Chemical Components)

- Convert ingredient lists into numerical vectors using **word embedding techniques**.
- Capture semantic relationships between chemical components.

Step 4: Similarity Analysis

- Compute similarity between cosmetic products based on ingredient embeddings.
- Identify products with chemically similar compositions.

Step 5: Dimensionality Reduction

- Apply **t-SNE** to reduce high-dimensional embedding vectors to 2D space.
- Preserve local similarity patterns among products.

Step 6: Visualization

- Use **Bokeh** to create interactive visualizations of ingredient similarity.
- Enable hover, zoom, and selection for product exploration.

Step 7: Recommendation System

- Implement a **content-based recommendation approach**.
- Recommend products with ingredient profiles similar to known dry-skin-friendly products.

Step 8: Evaluation & Interpretation

- Validate recommendations based on ingredient overlap and known skincare principles.
- Interpret clusters and similarities.

2.2 Use Case Diagram (Textual Representation)

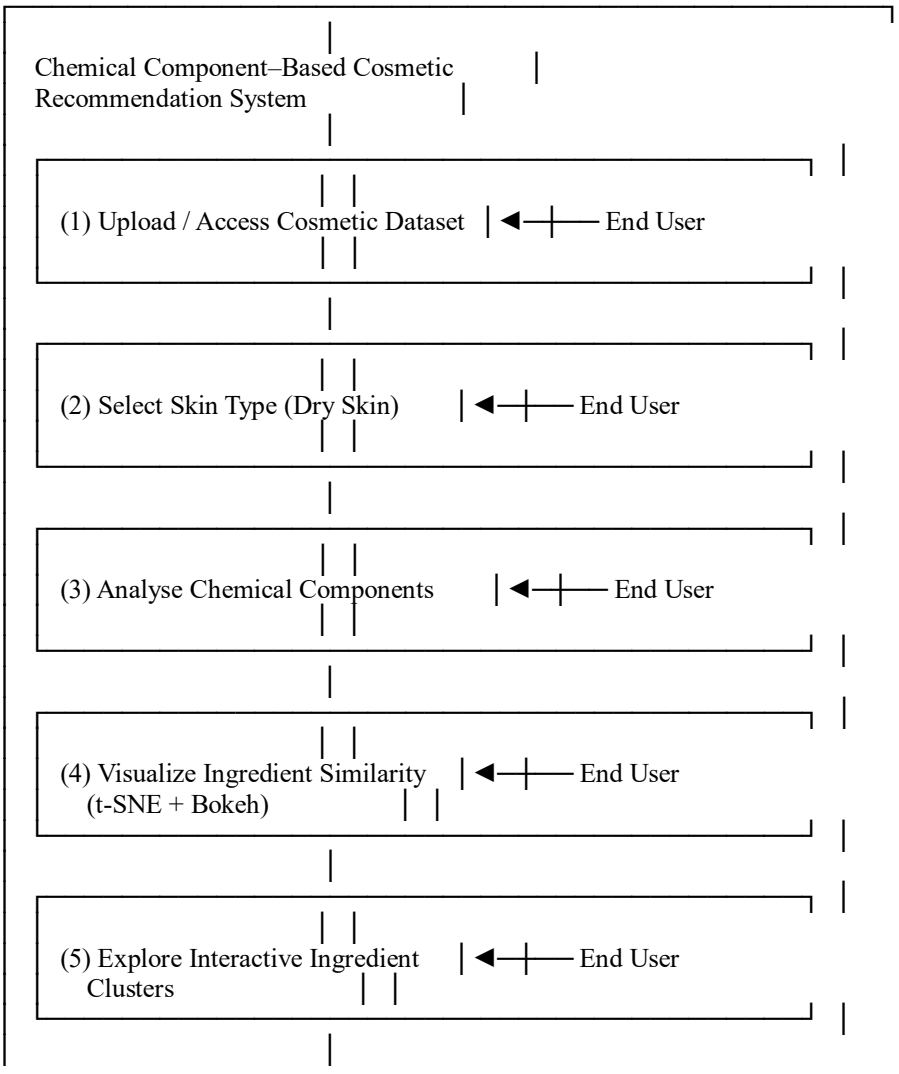
Actor

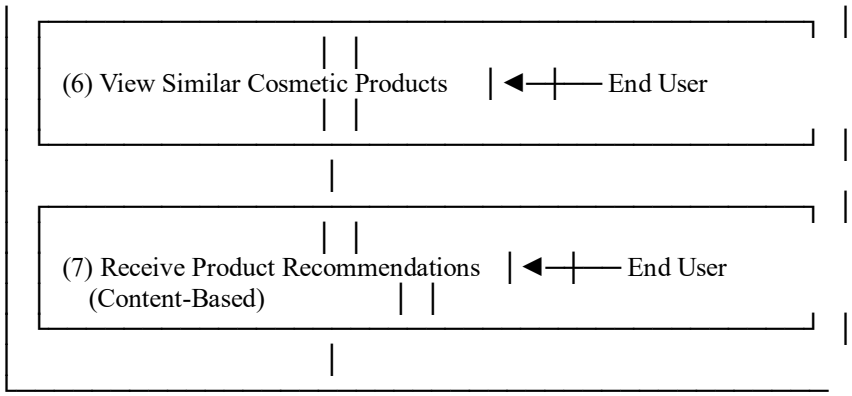
End User

(Consumer / Analyst)

System

Chemical Component-Based Cosmetics Recommendation System





2.3 Programming Language and Tools Used

Programming Language

Python is used as the primary programming language for this project due to its extensive ecosystem of libraries for data analysis, machine learning, and visualization. Python enables efficient handling of structured and unstructured data, making it well suited for processing cosmetic ingredient lists and performing chemical component–based analysis.

Dataset Format

The dataset is provided in **CSV (Comma-Separated Values)** format, which allows easy ingestion, manipulation, and preprocessing using Python data analysis libraries. The dataset contains cosmetic product information such as product name, brand, ingredient lists, labels, price, and supported skin types.

Python Libraries and Packages Used

1. Pandas

Purpose: Data loading, cleaning, filtering, and manipulation

Pandas is used to read the CSV dataset, handle missing values, filter cosmetic products based on category and skin type, and organize ingredient data into structured formats suitable for analysis.

2. NumPy

Purpose: Numerical computation and array operations

NumPy supports efficient numerical operations and is used for vectorized computations during feature engineering and similarity analysis.

3. Scikit-learn

Purpose: Machine learning and data preprocessing

Scikit-learn is used for:

- Text vectorization and feature encoding
- Dimensionality reduction using **t-SNE**
- Similarity computation between cosmetic products

4. Matplotlib

Purpose: Static data visualization

Matplotlib is used for exploratory visualizations such as ingredient frequency plots and price distributions to support data understanding and analysis.

5. Bokeh

Purpose: Interactive visualization

Bokeh is used to create interactive visualizations of ingredient similarity generated using t-SNE. It enables zooming, hovering, and exploration of product clusters based on chemical composition.

Development Environment

- **IDE:** Jupyter Notebook
- **Operating System:** Platform-independent

Summary of Technology Stack

Component	Technology
Programming Language	Python
Data Format	CSV
Data Processing	Pandas, NumPy
Machine Learning	Scikit-learn

Component	Technology
Visualization	Matplotlib, Bokeh
	Word Embeddings / TextVectorization

3. IMPLEMENTATION

3.1 Data observation and Data cleaning using required libraries

```
# import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE
```

+ Code + Text

```
df = pd.read_csv('/content/cosmetics.csv')
df.head(5)
```

	Label	Brand	Name	Price	Rank	Ingredients	Combination	Dry	Normal	Oily	Sensitive
0	Moisturizer	LA MER	Crème de la Mer	175	4.1	Algae (Seaweed) Extract, Mineral Oil, Petrolat...	1	1	1	1	1
1	Moisturizer	SK-II	Facial Treatment Essence	179	4.1	Galactomyces Ferment Filtrate (Pitera), Butyle...	1	1	1	1	1
2	Moisturizer	DRUNK ELEPHANT	Protini™ Polypeptide Cream	68	4.4	Water, Dicaprylyl Carbonate, Glycerin, Ceteary...	1	1	1	1	0
3	Moisturizer	LA MER	The Moisturizing	175	3.8	Algae (Seaweed) Extract,	1	1	1	1	1

Dataset first Observations

```
# find the shape of our cosmetic dataset

df.shape
```

```
(1472, 11)
```

```
# how many columns we have inside the whole dataset
df.columns
```

```
Index(['Label', 'Brand', 'Name', 'Price', 'Rank', 'Ingredients', 'Combination',
       'Dry', 'Normal', 'Oily', 'Sensitive'],
      dtype='object')
```

```
 # To get the basic information about the data types
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1472 entries, 0 to 1471
Data columns (total 11 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   Label        1472 non-null   object 
 1   Brand        1472 non-null   object 
 2   Name         1472 non-null   object 
 3   Price        1472 non-null   int64  
 4   Rank         1472 non-null   float64
 5   Ingredients  1472 non-null   object 
 6   Combination  1472 non-null   int64  
 7   Dry          1472 non-null   int64  
 8   Normal       1472 non-null   int64  
 9   Oily         1472 non-null   int64  
 10  Sensitive    1472 non-null   int64  
dtypes: float64(1), int64(6), object(4)
memory usage: 126.6+ KB
```

```
# Statistical data discription
df.describe()
```

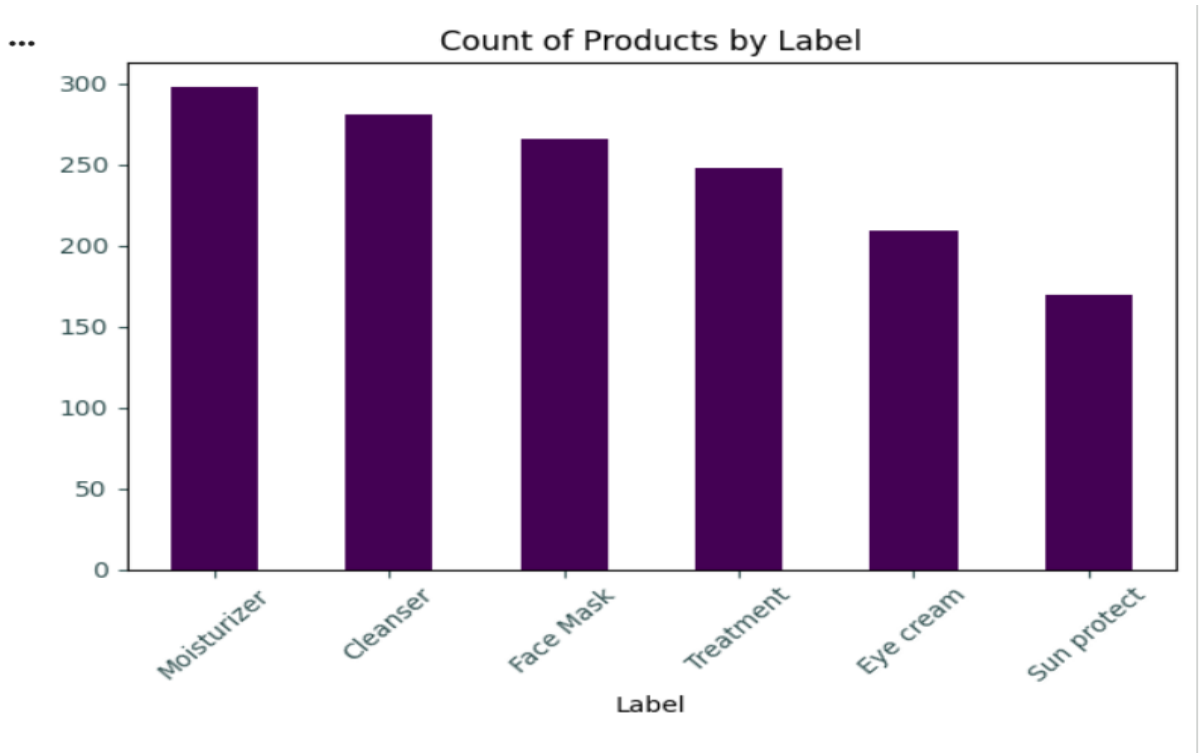
	Price	Rank	Combination	Dry	Normal	Oily	Sensitive
count	1472.000000	1472.000000	1472.00000	1472.000000	1472.000000	1472.000000	1472.000000
mean	55.584239	4.153261	0.65625	0.614130	0.652174	0.607337	0.513587
std	45.014429	0.633918	0.47512	0.486965	0.476442	0.488509	0.499985
min	3.000000	0.000000	0.00000	0.000000	0.000000	0.000000	0.000000
25%	30.000000	4.000000	0.00000	0.000000	0.000000	0.000000	0.000000
50%	42.500000	4.300000	1.00000	1.000000	1.000000	1.000000	1.000000
75%	68.000000	4.500000	1.00000	1.000000	1.000000	1.000000	1.000000
max	370.000000	5.000000	1.00000	1.000000	1.000000	1.000000	1.000000

Types of products and count

```
# find types of products and their count
df['Label'].value_counts()
```

Label	count
Moisturizer	298
Cleanser	281
Face Mask	266
Treatment	248
Eye cream	209
Sun protect	170

dtype: int64



3.2 Data Filtration

```
# create filter for moisturizers
moisturizers = df['Label'] == 'Moisturizers'
```

```
moisturizers = df[df['Label'].str.contains('Moisturizer')]
```

```
display(moisturizers.head())
```

	Label	Brand	Name	Price	Rank	Ingredients	Combination	Dry	Normal	Oily	Sensitive
0	Moisturizer	LA MER	Crème de la Mer	175	4.1	Algae (Seaweed) Extract, Mineral Oil, Petrolat...	1	1	1	1	1
1	Moisturizer	SK-II	Facial Treatment Essence	179	4.1	Galactomyces Ferment Filtrate (Pitera), Butyle...	1	1	1	1	1
2	Moisturizer	DRUNK ELEPHANT	Protini™ Polypeptide Cream	68	4.4	Water, Dicaprylyl Carbonate, Glycerin, Ceteary...	1	1	1	1	0
3	Moisturizer	LA MER	The Moisturizing Soft Cream	175	3.8	Algae (Seaweed) Extract, Cyclopentasiloxane, P...	1	1	1	1	1
4	Moisturizer	IT COSMETICS	Your Skin But Better™ CC+™ Cream with SPF 50+	38	4.1	Water, Snail Secretion Filtrate, Phenyl Trimet...	1	1	1	1	1

FOCUS ON DRY SKIN TYPE ONLY

```
# create filter moisturizer for dry skin type  save in moisturizer_dry
moisturizers_dry = moisturizers[moisturizers['Dry'] == 1]
```

```
moisturizers_dry.shape
```

```
(190, 11)
```

```
moisturizers_dry.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 190 entries, 0 to 297
Data columns (total 11 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   Label        190 non-null    object 
 1   Brand        190 non-null    object 
 2   Name         190 non-null    object 
 3   Price        190 non-null    int64  
 4   Rank         190 non-null    float64
 5   Ingredients  190 non-null    object 
 6   Combination  190 non-null    int64  
 7   Dry          190 non-null    int64  
 8   Normal       190 non-null    int64  
 9   Oily         190 non-null    int64  
 10  Sensitive    190 non-null    int64  
dtypes: float64(1), int64(6), object(4)
memory usage: 17.8+ KB
```

3.3 **Tokenizing the ingredients**

```
# Initialize dictionary,lists and initial index
corpus = []
ingredient_idx = {}
idx = 0


# for loop for tokenization
for i in range(len(moisturizers_dry)):
    ingredients = moisturizers_dry['Ingredients'][i]
    ingredients_lower = ingredients.lower()
    tokens = ingredients_lower.split(',')
    corpus.append(tokens) # Corrected to append the list of tokens
    for ingredient in tokens:
        # Strip whitespace from the ingredient string before using it as
        cleaned_ingredient = ingredient.strip()
        if cleaned_ingredient not in ingredient_idx:
            ingredient_idx[cleaned_ingredient] = idx # Corrected: use cle
            idx += 1
# check the result
print("The index for decyl oleate is ", ingredient_idx['decyl oleate'])
```

```
The index for decyl oleate is  25
```

3.4 Initializing a document -term matrix (DTM)

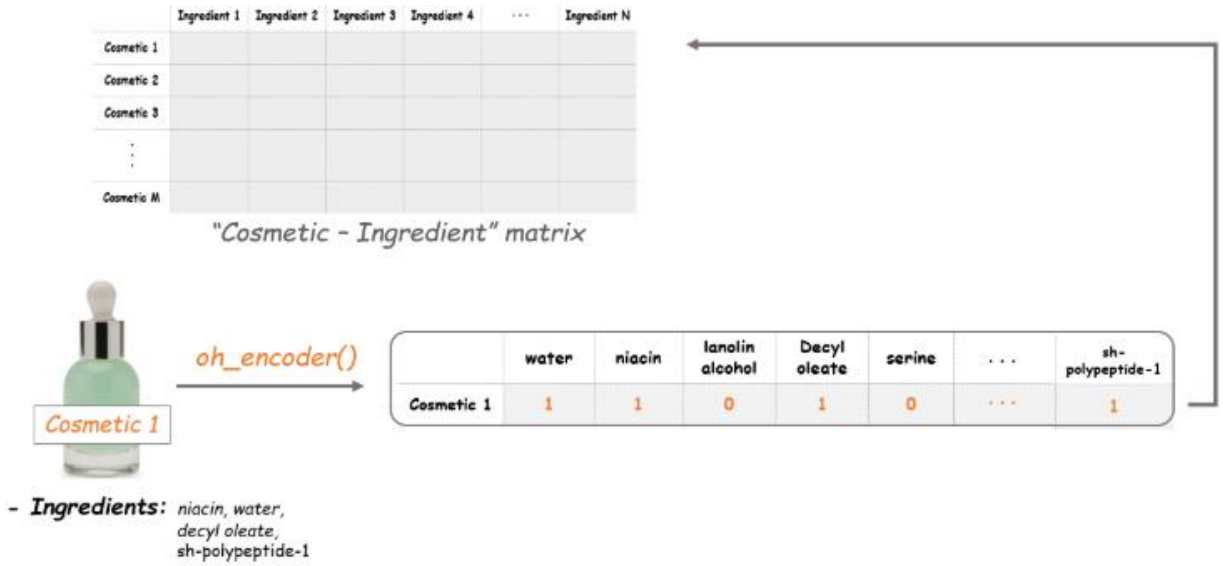

```
# get the number of items and tokens
M = len(corpus)
N = len(ingredient_idx)


# Initialize a matrix of zeros
A = np.zeros((M, N))
```

Creating a Counter Function to count the tokens

```
# Define the oh_encoder function
def oh_encoder(tokens):
    x = np.zeros(len(ingredient_idx))
    for ingredient in tokens:
        # Get the index for each ingredient
        idx = ingredient_idx[ingredient.strip()]
        # Put 1 at the corresponding indices
        x[idx] = 1
    return x
```

3.5 The Cosmetic – Ingredient matrix!




```
# Make a document-term matrix
i = 0
for tokens in corpus:
    A[i, :] = oh_encoder(tokens)
    i += 1
```

3.6 Time for Dimension reduction with t – SNE

```
# Dimension reduction with t-SNE
model = TSNE(n_components=2, random_state=0)
tsne_features = model.fit_transform(A)

# Make X, Y columns
moisturizers_dry['X'] = tsne_features[:, 0]
moisturizers_dry['Y'] = tsne_features[:, 1]
```

3.7 Let's map the items with Bokeh

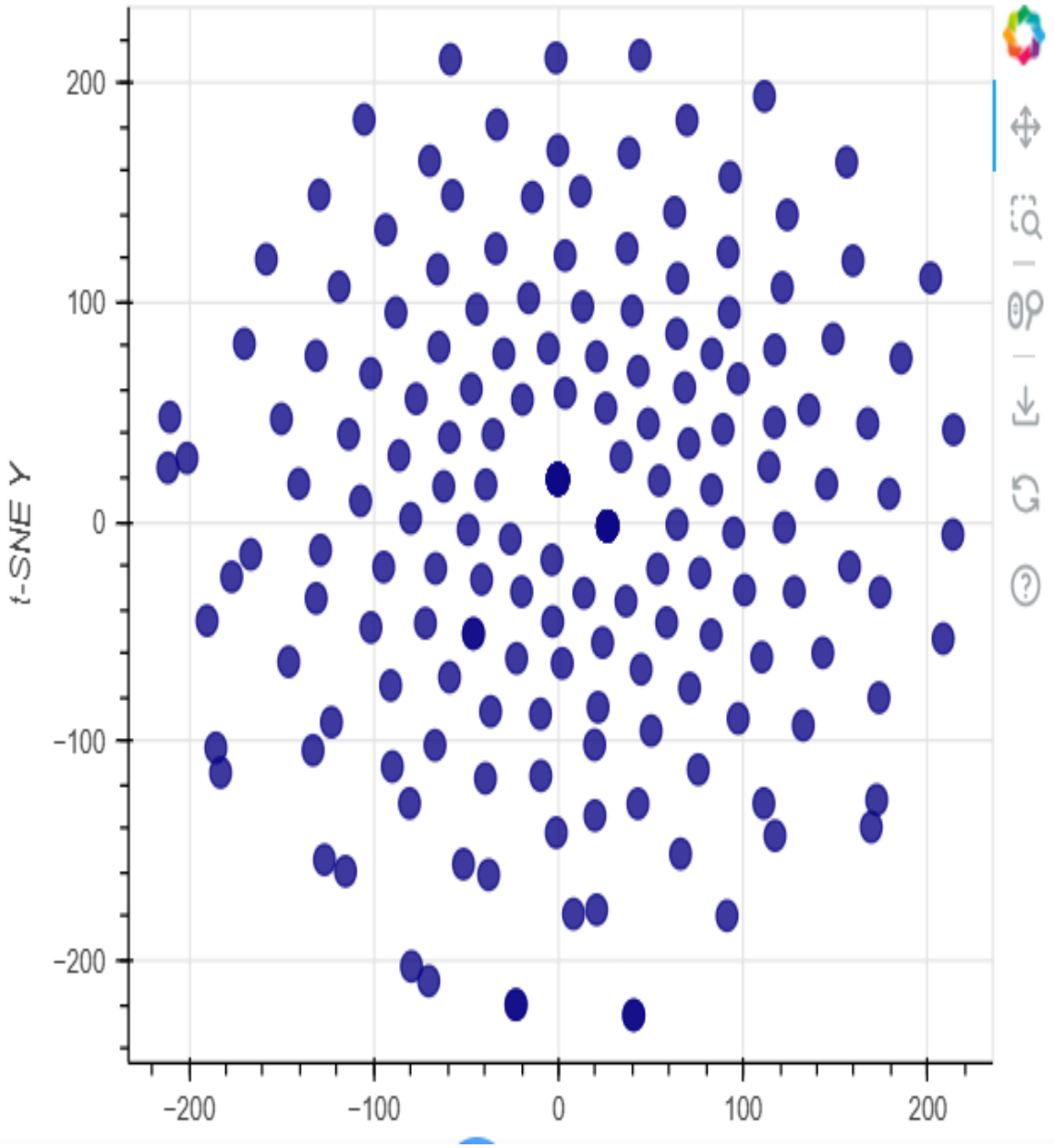
```
from bokeh.io import show, output_notebook, push_notebook
from bokeh.plotting import figure
from bokeh.models import ColumnDataSource, HoverTool
from bokeh.palettes import Plasma10 # Import the Plasma10 palette
output_notebook()

# Make a source and a scatter plot
source = ColumnDataSource(moisturizers_dry)

plot = figure(x_axis_label = 't-SNE X', # Label for the x-axis
              y_axis_label = 't-SNE Y', # Label for the y-axis
              width = 500, height = 400)
             # tools=[hover, 'pan', 'wheel_zoom', 'box_zoom', 'reset', 's

plot.scatter(x = 'X',
             y = 'Y',
             source = source,
             size = 10, color = Plasma10[0], alpha = .8) # Changed color
```

```
# Display the plot without tooltip
show(plot)
```



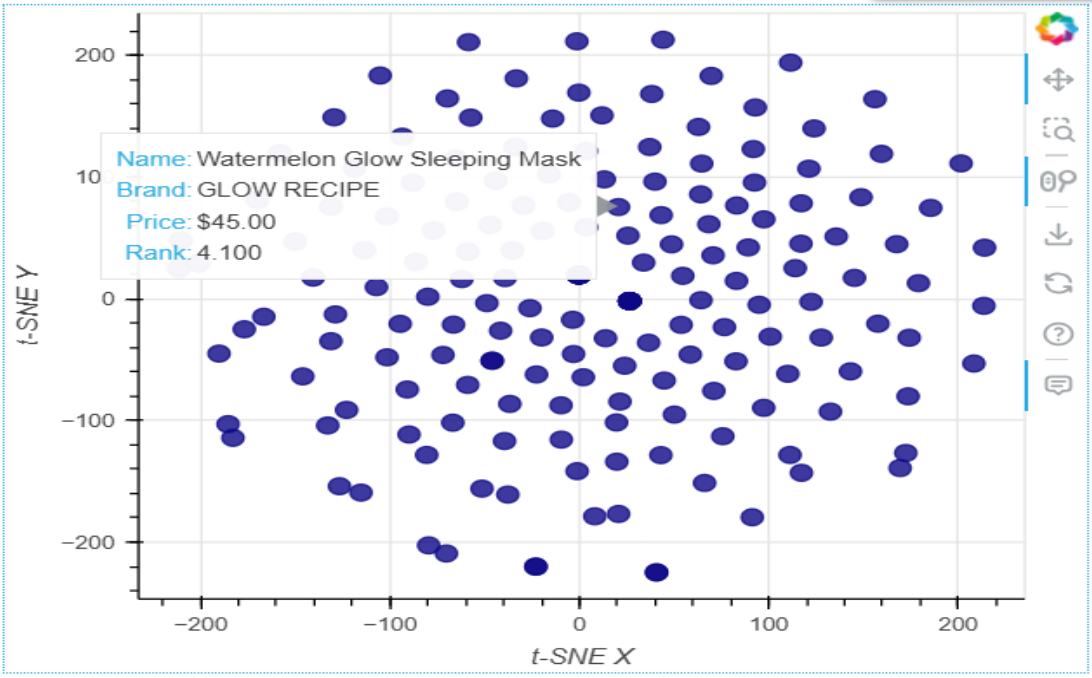
3.8 Adding a hover tool for tooltip

```
# Create a HoverTool object for tooltip
hover = HoverTool(tooltips = [("Name", "@Name"),
                              ("Brand", "@Brand"),
                              ("Price", "@Price{$0.00}"),
                              ("Rank", "@Rank")])

plot.add_tools(hover)
```

3.9 Finally Mapping the cosmetic items with Hover tool

```
show(plot)
```



On the t-SNE map, the distance between points is crucial for understanding ingredient similarity:
Closer points mean that the cosmetic items are more similar in their chemical composition. Further apart points indicate that the cosmetic items have less similar ingredient lists. So, if two products appear close together on the map, you can infer that they share a high degree of ingredient commonality, even if their brands or names are different. This allows for a visual comparison of cosmetics based purely on their ingredient profiles.

3.10 Comparing two products

```
# Print the ingredients of two similar cosmetics
cosmetic_1 = moisturizers_dry[moisturizers_dry['Name'] == "Color Control
cosmetic_2 = moisturizers_dry[moisturizers_dry['Name'] == "BB Cushion Hyd

# Display each item's data and ingredients
display(cosmetic_1)
print(cosmetic_1.Ingredients.values)
display(cosmetic_2)
print(cosmetic_2.Ingredients.values)
```

	Label	Brand	Name	Price	Rank	Ingredients	Combina
45	Moisturizer	AMOREPACIFIC	Color Control Cushion Compact Broad Spectrum S...	60	4.0	Phyllostachis Bambusoides Juice, Cyclopentasil...	

4 Conclusion of the Project

This project presents a data-driven approach to understanding cosmetic formulations for individuals with dry skin by analyzing chemical components and ingredient compositions. Using Python, CSV-based datasets, and Scikit-Learn, the system identifies ingredient patterns, measures similarity between products, and generates personalized recommendations.

The project successfully demonstrates that cosmetic products with similar ingredient chemistry can be matched scientifically, helping users discover safer and more suitable options for dry skin. It also offers transparency by visualizing chemical clusters, supporting informed decision-making for consumers, dermatology consultants, and product analysts.
This work establishes a solid foundation for ingredient-based recommendation systems in dermatological and cosmetic applications, highlighting how AI and chemical profiling can improve skincare product selection.

5. Recommendations

Based on the current project findings, the following recommendations are proposed:

1. **Prioritize Ingredients Over Branding**
    - Product selection should be guided primarily by chemical composition, not marketing claims.
2. **Promote Ingredient Transparency**
    - Manufacturers should provide clear, standardized ingredient listings to support AI-based matching.
3. **Use of Clustering for Safer Product Discovery**
    - Dermatologists and analysts can use cluster heatmaps to identify chemical families that benefit or harm dry skin.
4. **Continuous Dataset Expansion**
    - The dataset should be updated with new product launches, regulatory changes, and formulation improvements.

6. References

https://www.coursera.org/learn/data-analysis-with-python

https://www.geeksforgeeks.org/data-analysis/data-analysis-with-python/